

Analytic Underpinnings of the Symbolic Answer-Reduction Pipeline

4 September 2026

Abstract

The objects manipulated by answer reduction are finite descriptions of machines, circuits, formulas, polynomials, samplers, and verifiers. The Julia prototype realizes part of the corresponding analytic chain as typed transformations of explicit intermediate representations. It constructs finite-field polynomials and conditionally linear samplers, and it checks coefficient identities, degree accounts, dependency blocks, marginal laws, and finite-instance histograms. The operator-algebraic and compression theorems remain imported. This note makes the description-level Turing-machine-lambda-calculus correspondence precise, identifies the fixed point, and records the evidence boundary at every rung.

Contents

1	Objects, not functions	2
1.1	The paper’s intensional interface	2
1.2	A phase-separated lambda description language	3
2	The fixed point	4
2.1	Three finite descriptions	4
3	Cook–Levin as a transformation on descriptions	6
3.1	Bounded trace to five independent assignments	6
3.2	Tseitin and arithmetization	7
3.3	The occurrence-vector discrepancy	7
4	The low-degree PCP as algebra	8
4.1	Interpolation and the vanishing polynomial	8
4.2	Division by the Boolean ideal	8
4.3	PCP view and local verifier	9
4.4	The four soundness layers	9

5	The typed layer: conditionally linear functions	11
5.1	The inductive object and its laws	11
5.2	Point and line maps	11
5.3	Types and the six-copy sampler	12
5.4	The structural hypothesis	13
6	Correspondence tables	14
6.1	Description front end (planned TB3)	14
6.2	Polynomial rung (TB0)	14
6.3	CL and low-degree-test rung (TB1)	14
6.4	Typed answer-reduction rung (TB2)	15
6.5	Midpoint diagnostic (TB0.5)	15
6.6	Compression and fixed point (planned TB4)	16
7	What is analytic and what is computed	16
7.1	The evidence boundary	16
7.2	The midpoint diagnostic	17
7.3	Final accounting	17

1 Objects, not functions

1.1 The paper's intensional interface

Definition 1.1 (Decider, sampler, and normal form). A *decider* is a total five-input Turing machine $D(n, x, y, a, b) \in \{0, 1\}$. A *sampler* is a Turing machine exposing the dimension, marginal maps, conditional linear maps, and factor spaces of a question distribution. A *normal-form verifier* is the pair $V = (S, D)$, with the question distribution generated by S and the answer predicate computed by D . A normal-form verifier is λ -*bounded* when its two running times are at most n^λ for $n \geq 2$, and its description length is at most λ .

The definitions are literal machine definitions, not identifications up to extensional equality.¹ The sampler exposes the successive maps required by the marginal decomposition.² Normal form and λ -boundedness are respectively `def:normal-ver` and `def:lambda` in `ground-truth/gt-05-games-normalform.tex`.

The compression theorem accepts a pair of Turing machines and an integer, and returns a pair of Turing machines. Its hypotheses refer simultaneously to $|V|$, $\text{TIME}_S(n)$, and $\text{TIME}_D(n)$. Thus `Compress` cannot be a map on extensional values of the verifier: two programs computing the same predicate may have different description lengths and different time bounds. The input is a finite

¹Ground truth: `def:decider` in `ground-truth/gt-05-games-normalform.tex`.

²Ground truth: `def:sampler` in `ground-truth/gt-04-cl.tex`.

object that can be inspected, copied, hard-wired, and passed to a universal evaluator.³ The paper fixes a canonical Turing-machine encoding, permits polynomial simulation overhead, and separately accounts for hard-wired data.⁴

This is the operative distinction:

Object	What it contains	What may consume it
Closure	Code pointer plus runtime environment	Host evaluator only; it has no canonical project byte string
Quoted syntax	Closed syntax with canonical bytes	Specializer, compiler, universal evaluator, and Compress
Circuit	Finite Boolean DAG with named input blocks	Tseitin transformation and succinct clause evaluator
Universal evaluator	A fixed program interpreting quoted syntax	Bounded trace construction, with an explicit counter
Specialization	Code-to-code substitution of static data	Any later description transformation; it produces syntax, not a closure

The current design names the description sort `Quoted{A}`, quotation `Quote`, evaluation `Eval`, and code specialization `Specialize`. It gives `Quoted{A}` a canonical serialization and defines `description_size` as its byte length. These names are specified in `docs/definitions.md`; as of this document, they do not occur in `src/`. They are therefore a description-language specification, not a TB0–TB2 execution result.

1.2 A phase-separated lambda description language

Let $P\{A\}$ denote program syntax whose constructor contracts assign result sort A . The relevant grammar is

$$\begin{aligned}
P ::= & \text{BoundVar}(i, j) \mid \text{Hole}(h, A) \mid \text{Lambda}(r, P) \mid \text{Apply}(P, P^*) \\
& \mid \text{Fix}(P) \mid \text{If}(P, P, P) \mid \text{Prim}(p, B, P^*) \\
& \mid \text{Quote}(\text{Closed}(P)) \mid \text{Eval}(P_{\text{code}}, P^*, F) \mid \text{Specialize}(P_{\text{code}}, \rho).
\end{aligned}$$

Here a *hole* is a typed, static substitution site; it is not a dynamically scoped variable. F is either literal fuel or a symbolic finite fuel bound. `Eval` is total as a meta-operation: it returns a value, out-of-fuel, or a type error. Quotation requires closed syntax. Specialization requires a total sort-correct environment for exactly the outstanding holes. These phase conditions express the distinction between source, value, and compiled object that the machine presentation otherwise implements through encodings and universal simulation.⁵

Definition 1.2 (Description-language replacement). In this project, “Turing machine $\rightarrow$ lambda calculus” means choosing computable translations

$$\text{tm2q} : \text{TMDesc}_k \longrightarrow \text{Quoted}\{A\}, \quad \text{q2tm} : \text{Quoted}\{A\} \longrightarrow \text{TMDesc}_k$$

³Ground truth: `thm:compression` in `ground-truth/gt-12-compression.tex`.

⁴Ground truth: `sec:tms` in `ground-truth/gt-03-prelim.tex`.

⁵Ground truth: `thm:universal-tm` in `ground-truth/gt-03-prelim.tex`.

that preserve the computed partial function under the agreed encodings, with polynomial changes in description size and running time. Fuel T replaces the paper’s timeout counter; canonical byte length replaces $|D|$. No claim is made that the byte strings, small-step traces, or exact costs are identical.

Church–Turing equivalence alone would only say that the same partial functions are definable. It would not justify inserting a closure into **Compress**, would not provide $|D|$, and would not show that specializing a description has the resource bounds required by Cook–Levin. The needed statement is intensional and costed.

Lemma 1.3 (Fuel-bounded quoted evaluation is a bounded machine). *Fix a finite constructor set and a canonical prefix serialization of the grammar above. There are universal constants $C, c \geq 1$ and a Turing machine U_P , of constant description size, such that for every well-sorted quote d , encoded input tuple u , and fuel T , $U_P(d, u, T)$ returns the same value/out-of-fuel/type-error result as $\text{Eval}(d, u, T)$, and*

$$\text{TIME}_{U_P}(d, u, T) \leq C((|d| + |u| + 1)(T + 1) \log(T + 2))^c.$$

Hard-wiring d produces a machine $(U_P)_d$ with description length $O(|d| + 1)$ and the same polynomial time overhead. Both descriptions are computable from d in time polynomial in $|d|$.

Proof. Decode the prefix representation once and store the finite syntax tree, de Bruijn environment, continuation stack, and remaining-fuel counter on work tapes. Each abstract reduction changes a constant number of nodes and stack cells; conservative tree lookup and copying cost a polynomial in $|d| + |u| + T$. Decrementing a binary counter costs $O(\log(T + 2))$ per abstract step. After at most T reductions the simulator returns either the language result or out-of-fuel, so the displayed polynomial follows after absorbing representation choices into C, c . A standard single-tape universal machine adds only polynomial overhead.⁶ For the hard-wired form, the paper’s hardwiring construction gives a description of length linear in the embedded description and constructs it in linear, hence polynomial, time.⁷ The timeout-counter discussion in the same section gives the explicit $O(T \log T)$ counter overhead before universal-machine simulation. $\square$

Thus the costed translation preserves the asymptotic class of the hypotheses needed downstream in **prop:standard-succinct-sat** and **thm:compression**. This is an analytic lemma in this note; the current Julia tree has no quoted evaluator against which to replay it.

2 The fixed point

2.1 Three finite descriptions

Kleene’s second recursion theorem says, in the form needed here, that for an effective operation mapping program descriptions to program descriptions, there is an index whose computed behavior equals the behavior obtained by feeding that index to the operation. The untyped lambda term $Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$ expresses the same self-application extensionally. A general $Y : (A \rightarrow A) \rightarrow A$ is not typable in the simply typed lambda calculus. The typed adaptation is therefore

⁶Ground truth: **thm:universal-tm** in **ground-truth/gt-03-prelim.tex**.

⁷Ground truth: **sec:tms** in **ground-truth/gt-03-prelim.tex**.

not to smuggle untyped self-application into the value language: it is to add a description-level constructor `Fix` (also called `YCode` in the planned API) that binds one distinguished hole of sort `Quoted{A}`.

The paper implements the self-reference directly. Its machine $\mathcal{F}$ is invoked as $\mathcal{F}(\overline{\mathcal{F}}, \overline{M}, \lambda, n, x, y, a, b)$, builds the description of the five-input machine obtained by hard-wiring the first three entries, pairs that description with the sampler description, and gives the pair to `Compress`. The accompanying remark says that an earlier version instead used Kleene’s recursion theorem.⁸

In the quoted syntax, let $M : P\{\text{MachineDesc}\}$ and $L : P\{\text{Level}\}$ be closed literal terms, and let S_L and `Compress` be closed quoted programs of the indicated sorts. The design’s partial body is

$$\begin{aligned} C_{M,L}(h) &= \text{Apply}(\text{Quote}(\text{Compress}), \text{quoted_pair}(\text{Quote}(S_L), h), L), \\ \Psi_{M,L} &= \text{Lambda}(5, \text{If}(\text{Prim}(\text{halts_within}, M, n), \text{Prim}(\text{true}), \\ &\quad \text{Eval}(C_{M,L}(\text{Hole}(\text{self_code}, \text{Quoted}\{\text{Decider}\})), \\ &\quad (n, x, y, a, b), \text{FuelBound}(n, L))))). \end{aligned}$$

Writing $d_{M,L} = \text{Quote}(\text{Fix}(\Psi_{M,L}))$, the defining description equation is

$$\text{Specialize}(\Psi_{M,L}, \{\text{self_code} \mapsto d_{M,L}\}) \quad \text{and} \quad D_{M,L} = Y_{\text{code}} \Psi_{M,L} = \text{Fix}(\Psi_{M,L}).$$

Operationally this says

$$\text{Eval}(d_{M,L}, u; T) = \text{Eval}(\text{Quote Specialize}(\Psi_{M,L}, \{\text{self_code} \mapsto d_{M,L}\}), u; T)$$

whenever both sides reach a value within the compared fuel convention. The display is a code equation: $d_{M,L}$ is a finite byte string even though its evaluation may recursively request its own bytes.

Proposition 2.1 (Equivalence of the fixed-point presentations). *Assume canonical encodings and effective, capture-free hardwiring. The following presentations are intertranslatable by total transformations on finite descriptions, with polynomial size and construction overhead:*

1. the paper’s explicit $\mathcal{F}(\overline{\mathcal{F}}, \overline{M}, L, \dots)$ self-application;
2. a Kleene fixed-point index for the description transformer induced by $\Psi_{M,L}$;
3. the quoted term $\text{Fix}(\Psi_{M,L})$, whose `self_code` hole is specialized with its own quote.

The translations preserve the role of self-description: they produce finite codes that can be measured and passed to `Compress`, not merely functions that happen to agree.

Proof. For (1) to (2), regard the code-building portion of $\mathcal{F}$ as an effective transformer $f(e)$ that hard-wires $e, \overline{M}, L$ and returns a five-input decider description. Kleene’s theorem produces an index e_0 with $\varphi_{e_0} = \varphi_{f(e_0)}$; the index itself is the finite object inserted into the verifier description. For (2) to (3), implement the standard diagonal construction inside `Fix`: canonical quotation supplies the index and capture-free `Specialize` implements hardwiring. For (3) to (1), compile the quoted evaluator and specializer to a Turing machine by the previous lemma, and hard-wire the canonical bytes of the body. This yields an explicit machine that receives its own compiled description exactly

⁸Ground truth: `fig:halt_f` in `ground-truth/gt-12-compression.tex`.

where the hole occurred. Universal simulation and hardwiring give polynomial overheads in each direction.⁹ The paper independently records both the explicit construction and the earlier Kleene formulation at `fig:halt_f` in `ground-truth/gt-12-compression.tex`. $\square$

This is the easy part. `Compress` must traverse the quoted body, compute its size, and specialize code-producing subroutines; it must not evaluate the body merely to recover an extensional predicate. The fixed point gives self-reference. It supplies none of the gap preservation, low-degree soundness, or rigidity conclusions of the compression theorem; here *rigidity* means the operator statement that observed correlations force a controlled common measurement structure.

3 Cook–Levin as a transformation on descriptions

3.1 Bounded trace to five independent assignments

For a quoted decider D , fixed inputs (n, x, y, a, b) , and fuel T , a *bounded trace* is the finite list of configurations produced before acceptance, rejection, or timeout. Cook–Levin converts the local consistency conditions of this trace into a 3SAT instance. The version used here returns a small circuit deciding whether an indexed signed clause belongs to an exponentially larger formula. Its size is polynomial in $\log n, \log T, Q, \sigma$, with $Q \leq T$ and $|D| \leq \sigma$.¹⁰ This is already an analytic source-to-source transformation: the output is a description of a clause relation, not an enumerated truth table.

The paper then removes two obstructions to the later query pattern. First, a 3SAT clause addresses three locations of one assignment, whereas the low-degree enforcement can return values consistent with three separate assignments. Second, the two prover answers a, b occur as substrings of a larger trace assignment, making a uniform restricted query cumbersome under conditional linearity. The remedy is a succinct decoupled 5SAT relation with five separately indexed blocks

$$a_{i_1}^{o_1} \vee b_{i_2}^{o_2} \vee u_{3,i_3}^{o_3} \vee u_{4,i_4}^{o_4} \vee u_{5,i_5}^{o_5}.$$

The first two blocks expose a, b directly; the last three need not equal one another.¹¹ The general transformation and its padded equal-block form are ground-truth constructions; no `bounded_trace`, `cook_levin`, or `decouple5` implementation is currently present in `src/`.

The intended IR chain is

$$\begin{aligned} \text{Quoted}\{\text{Decider}\} &\xrightarrow{\text{bounded_trace}} \text{BoundedTrace} \xrightarrow{\text{cook_levin}} \text{Succinct3SAT} \\ &\xrightarrow{\text{decouple5}} \text{SuccinctDecoupled5SAT} \\ &\xrightarrow{\text{padding}} \text{PaddedSuccinctDecoupled5SAT}. \end{aligned}$$

At present these names describe planned transformations from `docs/definitions.md`; their analytic contracts are CITED.

⁹Ground truth: `sec:tms` in `ground-truth/gt-03-prelim.tex`.

¹⁰Ground truth: `prop:standard-succinct-sat` in `ground-truth/gt-10-answer-reduction.tex`.

¹¹Ground truth: `sec:succinct-deciders` in `ground-truth/gt-10-answer-reduction.tex`.

3.2 Tseitin and arithmetization

Given a Boolean circuit C with gates g_i , Tseitin introduces a wire variable w_i and an equivalence formula

$$z_i = (g_i(x, w) \wedge w_i) \vee (\neg g_i(x, w) \wedge \neg w_i), \quad F = \bigwedge_i z_i.$$

The required contract is $C(x) = 1$ iff there exists w with $F(x, w) = 1$, with size and construction time $O(|C|)$.¹² The project follows the explicit gate-equivalence form in `ground-truth/nw19/nw19-tseitin-arith.tex` and also conjoins the output literal w_{out} , so the stored formula explicitly asserts acceptance. That convention is marked `SOURCE_REPAIR`; it is not silently attributed to the source.

The Julia constructors `Circuit`, `Input`, `Gate`, `NotGate`, `AndGate`, and `OrGate` make the finite DAG and its topological order explicit. The function `tseitin` returns `Checked{TseitinFormula, CertNode}`. Its replay recomputes the formula’s occurrence vector; it does not, by that replay alone, establish the general Cook–Levin or Tseitin semantic equivalence. Those implications are exhausted only on the small TB0 fixture and remain cited in general.

Arithmetization recursively applies

$$\neg A \mapsto 1 - A, \quad A \wedge B \mapsto AB, \quad A \vee B \mapsto A + B - AB.$$

It agrees with the Boolean formula on the Boolean cube.¹³ The related NW19 rule is `def:arithmetization` in `ground-truth/nw19/nw19-tseitin-arith.tex`. Julia’s `arith_q` returns a sparse formal Poly; its `ArithTseitin` certificate checks support degrees against a structural bound.

3.3 The occurrence-vector discrepancy

There is a discrepancy in our reading of the individual-degree bookkeeping; our reading may be wrong. The source proposition states individual degree at most two for the Tseitin arithmetization.¹⁴ Reading the NW19 formula literally as a formula tree, arithmetization is multilinear in leaf occurrences, not necessarily in circuit wires. The same wire can occur in several gate gadgets. With the project’s explicit output literal, the count we compute is

$$\text{occ}(x_j) = 2 \text{fanout}(x_j), \quad \text{occ}(w_i) = 2 + 2 \text{fanout}(w_i) + \mathbf{1}_{i=\text{out}},$$

and structural induction gives

$$\deg_v(F_{\text{arith}}) \leq \text{occ}_v(F).$$

The function `tseitin_occurrence_account` computes this vector; `occurrences` independently traverses the formula; and `arith_q` compares the structural account with actual sparse support.

Claim C8 is only C8: TESTED: the two-gate regression gives $(2, 2, 2, 4, 3)$, and the 16-variable TB0 formula attains

$$(2, 0, 0, 0, 2, 2, 0, 0, 0, 0, 6, 4, 4, 4, 4, 3).$$

The general occurrence formula is a written derivation, not a machine theorem. The downstream soundness repair is only C5: SKETCH. With a symbolic bound $\delta_F = \max_v \text{occ}_v(F)$, the conservative

¹²Ground truth: `def:tseitin` in `ground-truth/gt-10-answer-reduction.tex`.

¹³Ground truth: `def:formula-arithmetization` in `ground-truth/gt-10-answer-reduction.tex`.

¹⁴Ground truth: `prop:tseitin-arith-degree` in `ground-truth/gt-10-answer-reduction.tex`.

formula-test term becomes $(\delta_F + 5d)m'/q$, and `P_formula_structural` is an additional obligation. Under an explicit fanout-reduction hypothesis, $\text{fanout}_{\max} \leq 2$ gives $\delta_F \leq 7$ with the output literal, and $d \geq 8$ suffices for the individual-degree construction bound. Absorption into the asymptotic parameter choice still needs the stated growth hypotheses; it is not promoted by the two finite computations.

4 The low-degree PCP as algebra

4.1 Interpolation and the vanishing polynomial

For $M = 2^m$ and $a = (a_y)_{y \in \{0,1\}^m}$, define

$$\text{ind}_{m,y}(x) = \prod_{j:y_j=1} x_j \prod_{j:y_j=0} (1 - x_j), \quad g_a(x) = \sum_y a_y \text{ind}_{m,y}(x).$$

Then $g_a(y) = a_y$ on the Boolean cube and $a \mapsto g_a$ is linear. This is the low-degree code: the assignment is represented by the evaluation table of its unique multilinear extension.¹⁵ The Julia function `g_a` constructs the sparse polynomial; block coordinates are carried by `VarLayout` and `VarBlock`; `dependency_blocks` recomputes the actual support dependency.

For five satisfying blocks, let g_i be their multilinear extensions and write $z = (x_1, \dots, x_5, o, w) \in \mathbb{F}_q^{m'}$. The central analytic object is

$$c_0(z) = F_{\text{arith}}(z) \prod_{i=1}^5 (g_i(x_i) - o_i).$$

On a Boolean point, either $F_{\text{arith}} = 0$, or the indicated clause is present and at least one factor $g_i(x_i) - o_i$ vanishes. Hence c_0 vanishes on $\{0,1\}^{m'}$. The code executes the product in `build_c0`; the `BuildC0` certificate replays the structural and actual degree accounts. The implication from a satisfying decoupled instance to Boolean-cube vanishing is checked only on the TB0 fixture and cited in general.¹⁶

4.2 Division by the Boolean ideal

Let $z_i(1 - z_i) = z_i - z_i^2$. The ground-truth basis proposition says that a degree-bounded polynomial vanishing on the Boolean cube lies in the ideal generated by these m' polynomials.¹⁷ The implementation uses the following deterministic monomial rewrite.

Lemma 4.1 (Executable zero-basis rewrite). *For a coefficient a , a monomial u independent of the displayed power, and $e \geq 2$,*

$$auz_i^e \mapsto auz_i^{e-1} - auz_i^{e-2} z_i(1 - z_i).$$

Iterating in a fixed variable and monomial order yields polynomials c_i and a remainder r , multilinear in every variable, such that

$$c_0 = \sum_{i=1}^{m'} c_i z_i(1 - z_i) + r.$$

¹⁵Ground truth: `sec:ld-encoding` in `ground-truth/gt-03-prelim.tex`.

¹⁶Ground truth: `thm:pcp-decider` in `ground-truth/gt-10-answer-reduction.tex`.

¹⁷Ground truth: `prop:zero-basis` in `ground-truth/gt-10-answer-reduction.tex`.

If the coordinate degree vector of c_0 is D , then $\deg_j(c_i) \leq D_j$, $\deg_i(c_i) \leq \max(D_i - 2, 0)$, and after reducing coordinates $1, \dots, i - 1$, the running remainder has degree at most one in those coordinates.

Proof. Expanding the right-hand side of one rewrite gives

$$auz_i^{e-1} - auz_i^{e-2}(z_i - z_i^2) = auz_i^e.$$

Thus every step preserves the coefficient polynomial after adding the recorded quotient contribution. It lowers the current exponent to one and places power $e - 2$ in the quotient, proving the degree bounds. Induction over the ordered coordinates gives the displayed decomposition and a final multilinear remainder. If c_0 vanishes on the Boolean cube, then so does r ; uniqueness of multilinear interpolation on $\{0, 1\}^{m'}$ makes $r = 0$. This is the constructive division used in the proof of `prop:zero-basis` in `ground-truth/gt-10-answer-reduction.tex`. $\square$

`zero_basis_decompose` stores every step as `RewriteStep` inside a `ZeroDecomposition`. `verify_rewrite_step` replays the scalar identity, and `verify_zero_decomposition` reconstructs the entire right side as a normalized coefficient dictionary. Promotion to a zero certificate also requires empty remainder. This is stronger than checking the equality at sampled points, but it remains an instance certificate, not a proof of the PCP soundness theorem.

4.3 PCP view and local verifier

A *PCP view* at z is

$$\text{ev}_z(\Pi) = (g_1(x_1), \dots, g_5(x_5), c_0(z), c_1(z), \dots, c_{m'}(z)).$$

The proof object is the tuple of evaluation tables for those polynomials. The labels are `def:pcp-proof` and `def:pcp-eval` in `ground-truth/gt-10-answer-reduction.tex`. Julia represents them by `PCPProof` and `PCPView`; `ev_z` refuses a g_i whose actual support escapes block X_i .

The verifier first reconstructs the succinct circuit, Tseitin formula, and arithmetization. It then checks exactly

$$\beta_0 = F_{\text{arith}}(z) \prod_{i=1}^5 (\alpha_i - o_i), \quad \beta_0 = \sum_{j=1}^{m'} \beta_j z_j (1 - z_j).$$

These are the formula test and zero-on-subcube test.¹⁸ The Julia function `pcpverifier` executes these two equations on an already supplied `TseitinFormula` and `view`. In TB2 it does not implement the figure's preceding reconstruction steps; the formula is construction-time data. `build_pcp` combines upstream certificates, coefficient division, degree checking, and stored certified views.

4.4 The four soundness layers

The four layers must not be collapsed.

¹⁸Ground truth: `fig:pcpverifier` in `ground-truth/gt-10-answer-reduction.tex`.

1. Algebraic soundness under a low-degree premise. Assume every submitted g_i, c_j has individual degree at most d . If the formula-test polynomials are unequal, their difference has total degree at most $(\delta_F + 5d)m'$ under the occurrence-vector account. If the zero-test polynomials are unequal, their difference has total degree at most $(2 + d)m'$. Schwartz–Zippel states that unequal polynomials of total degree at most Δ agree at a uniform point of $\mathbb{F}_q^{m'}$ with probability at most Δ/q .¹⁹ Consequently

$$\Pr[\text{false formula identity}] \leq \frac{(\delta_F + 5d)m'}{q}, \quad \Pr[\text{false zero identity}] \leq \frac{(2 + d)m'}{q}.$$

The paper’s displayed first numerator uses $2 + 5d$; the structural project obligation substitutes the measured δ_F under the discrepancy stated above. The field size enters nowhere mysteriously: $q = 2^k$ is chosen so both fractions are below the acceptance threshold $1/2$, together with the other PCP parameter obligations. The first structural inequality is extra to the literal `def:pcpparams` predicate in the present project.

If acceptance is greater than $1/2$ and both upper bounds are below $1/2$, each sampled equality is therefore a polynomial identity. The zero identity makes c_0 vanish on the Boolean cube; the formula identity then decodes five Boolean assignments satisfying the decoupled formula, and hence an accepting bounded computation. This conditional theorem is `thm:pcp-decider` in `ground-truth/gt-10-answer-reduction.tex`. Its current project status is C5: SKETCH, not CHECKED.

2. Enforcement of the low-degree premise. The simultaneous low-degree game queries points, axis-parallel lines, and diagonal lines. Its classical decider checks consistency of a point value with a claimed line restriction.²⁰ The theorem extracting approximately consistent low-individual-degree polynomial measurements from a successful quantum strategy is `lem:ld-soundness` in `ground-truth/gt-07-ldt.tex`. It is CITED. Running `ld_decider` on honest finite inputs does not prove this extraction theorem.

3. Polynomial identity from random evaluation. This is exactly the Schwartz–Zippel step above, used twice with two different degree bounds. The local program checks evaluations. The analytic theorem turns an acceptance probability into formal identity, provided the degree and field-size premises hold. No sample count can replace that implication.

4. Quantum consistency and rigidity. Answer reduction must connect different players’ point and line answers to the same polynomial measurements, control disturbance, and transport the result back to the original verifier. Those operator estimates use low-degree soundness, typed consistency transformations, and later rigidity arguments. They are imported in the soundness contract of `thm:ar` in `ground-truth/gt-10-answer-reduction.tex`; none is a sparse-polynomial identity.

The computer adds exactly this: on a named instance it constructs the polynomials, executes the division, compares normalized coefficients, computes degree and dependency vectors, and evaluates the two local equations. It does not test the theorem. Current statuses reflect that separation: C3 is TESTED, C5 is SKETCH, C1 and C2 remain CONJECTURE, and C8 is TESTED only on its two named circuits.

¹⁹Ground truth: `lem:schwartz-zippel` in `ground-truth/gt-03-prelim.tex`.

²⁰Ground truth: `fig:ld-decider` in `ground-truth/gt-07-ldt.tex`.

5 The typed layer: conditionally linear functions

5.1 The inductive object and its laws

Definition 5.1 (Conditionally linear function). Let $V = \mathbb{F}^n$ with fixed coordinate-register subspaces. The unique level-zero conditionally linear (CL) function is zero. A level- ℓ CL function chooses complementary registers $V_1 \oplus V_{>1} = V$, applies a linear map $A : V_1 \rightarrow V_1$, and, conditional only on $A(x^{V_1})$, selects a level- $(\ell - 1)$ CL function on $V_{>1}$. Its output is the sum of the first-stage and continuation outputs.

This is `def:cl-func` in `ground-truth/gt-04-cl.tex`. Julia mirrors it with `AbstractCL`, `CLZero`, and `CLStep`. The constructor verifies disjoint factor/rest coordinate sets, a matrix acting inside the factor, a fixed child level, and exact coverage of the rest register. Thus an inhabitant of `CLStep` has a constructed level; linearity is carried by the matrix, not inferred from samples.

The marginal $L_{\leq k}$ is the sum of the first k stage outputs, together with their factor spaces and linear maps. This is the decomposition of `lem:cl-kth` in `ground-truth/gt-04-cl.tex`. Julia's `marginal_k` returns a `CLMarginal`; `sum_stage_outputs` recomputes its value.

Lemma 5.2 (Level laws). *Let L be level k , let every continuation R_u be level ℓ , and let L_j act on pairwise disjoint registers at levels ℓ_j . Then*

$$\text{level}(\text{concatenate}(L, R)) = k + \ell, \quad \text{level}\left(\bigoplus_j L_j\right) = \max_j \ell_j.$$

For CL distributions S_1, S_2 , their product, formed by direct-summing the left maps and the right maps on independent seed registers, has level the maximum of the two levels.

Proof. For concatenation, induct on the outer constructor of L . At level zero, grafting R_0 supplies exactly ℓ stages. At a `CLStep`, retain its first linear stage and apply the induction hypothesis to each child; this gives $1 + (k - 1 + \ell) = k + \ell$. This is the construction in `lem:cl-concat` of `ground-truth/gt-04-cl.tex`.

For direct sum, pad each component by zero stages to $r = \max_j \ell_j$. At stage i , use the direct sum of the components' stage matrices on the direct sum of their factor registers. The continuation depends only on the tuple of earlier outputs, hence remains CL. Induction on r proves the law, matching `lem:cl-func-prod` in `ground-truth/gt-04-cl.tex`. A distribution product applies this law separately to its left and right maps, so its level is the same maximum. $\square$

The code functions are `concatenate`, `direct_sum`, `distribution`, and `product`. Their type constructions carry the level laws. The stronger structural hypothesis that all compiler operations of interest close inside one useful algebra is not implied by these individual lemmas.

5.2 Point and line maps

For $V = V_{\text{pt}} \oplus V_{\text{coord}} \oplus V_{\text{dir}}$, write a seed as $(u, s, v) \in \mathbb{F}_q^m \times \mathbb{F}_q \times \mathbb{F}_q^m$. Let L_v^{lf} be the canonical projection whose kernel is $\text{span}(v)$, so $L_v^{\text{lf}}u$ is a canonical representative of the affine line $u + \mathbb{F}_q v$.

²¹ The line-representative use is `def:line-representative` in `ground-truth/gt-07-ldt.tex`. Define $\chi(s)$ by splitting the fixed integer representatives of $\mathbb{F}_q$ into m equal buckets; this requires $m \mid q$. With π_{i-1} zeroing the first $i-1$ coordinates,

$$\begin{aligned} L_{\text{Point}}(u, s, v) &= (u, 0, 0), \\ L_{\text{ALine}}(u, s, v) &= (L_{e_{\chi(s)}}^{\text{lnf}} u, s, 0), \\ L_{\text{DLine}}(u, s, v) &= (L_{\pi_{\chi(s)-1}(v)}^{\text{lnf}} u, s, \pi_{\chi(s)-1}(v)). \end{aligned}$$

Their levels are respectively one, two, and three by their stage factorizations.²² The implementation names are `L_Point`, `L_ALine`, `L_DLine`, `L_lnf`, `chi`, and `pi_prefix`. Since the source’s kernel-basis definition does not cover $v = 0$, the code uses $L_0^{\text{lnf}} = I$ and marks the convention `SOURCE_REPAIR`.

The distribution $\mu_{L,R}$ pushes one uniform shared seed through the two maps.²³ For $(q, m) = (8, 2)$, TB1 enumerates all $8^5 = 32768$ seeds. The axis-line/point histogram has support 512, and the diagonal-line/point histogram has support 18432, including the zero directions. Both agree entry-for-entry with separately transcribed formulas for `lem:alnf` and `lem:dlnf` in `ground-truth/gt-07-ldt.tex`; every marginal is replayed. This finite statement is **C4a**: TESTED. It is not a statistical argument and does not establish quantum low-degree soundness.

5.3 Types and the six-copy sampler

A *typed sampler* is a finite type-graph-indexed family of CL maps sharing one ambient seed space. It first samples an oriented type edge and then pushes one uniform seed through the selected left/right maps.²⁴ Its induced distribution is `def:typed-sampler-sample` in the same file. Julia’s `TypedSampler` checks exact type coverage, a common field and seed dimension, graph endpoints, and a padded common level.

Detyping encodes the external graph choice into an untyped CL distribution by a rejection-sampling construction. It adds two levels, preserves value-one completeness, and changes soundness error by $16^{|\text{Type}|}$.²⁵ The Julia function `detype` returns `CitedDetypedVerifier`; the certificate is deliberately CITED. For the 54 answer-reduction types, the displayed factor is 16^{54} , not an executed estimate.

The answer-reduction PCP sampler has the 18 types

$$\{\text{Point}_i, \text{ALine}_i, \text{DLine}_i : 1 \leq i \leq 6\}$$

and a complete type graph. Copies 1–5 query one m -variate g_i each; copy 6 queries the bundle $(g_1, \dots, g_5, c_0, \dots, c_{m'})$ in dimension m' . The six-copy construction and its ambient direct sums are in `sec:ld-compiler` of `ground-truth/gt-10-answer-reduction.tex`. The current code uses `PCPType`, `PCPRegisterLayout`, `PCPCLMap`, `PCPPaddedMap`, and `pcp_sampler`. It carries a visible `SOURCE_REPAIR` for the copy-6 coordinate-register conflict between the displayed ambient space and the parser table.

Oracularization is the typed transformation in which the isolated roles receive the original left or right CL image while an oracle role receives the common seed and can reconstruct both; the code name

²¹Ground truth: `def:cl-canonical` in `ground-truth/gt-03-prelim.tex`.

²²Ground truth: `sec:ld-game` in `ground-truth/gt-07-ldt.tex`.

²³Ground truth: `def:cl-dist` in `ground-truth/gt-04-cl.tex`.

²⁴Ground truth: `def:typed-sampler` in `ground-truth/gt-06-types.tex`.

²⁵Ground truth: `lem:detyping-verifiers` in `ground-truth/gt-06-types.tex`.

is `oracularize_sampler`. Its theorem contract is CITED.²⁶ The typed answer-reduced sampler is the product of this three-role family with the 18-type PCP family. `typed_sampler_product` produces 54 types and combines levels by a maximum. The code constructs the product and checks its shape, but C4b remains CONJECTURE: the latest TB2 critic held promotion because the lazy PCPCLMap’s stage depth was not itself enough to make the full CL decomposition a constructor-level fact. Local certificate labels must not be confused with a promoted claim.

The typed decider implements the five guarded checks of `fig:decider-pcp`: global consistency, input consistency, input low degree, proof encoding (individual and simultaneous low degree), and the PCP game check.²⁷ The Julia names are `TypedAnswerReducedDecider`, `typed_answer_reduced_decider`, `LDParams`, `ld_decider`, and `PCPGameCall`. `answer_reduce_pcp` constructs the typed verifier of level $\max(\ell, 3)$ on its fixture and attaches explicit assumptions.

The full theorem contract is conditional:

ASSUME: V is an ℓ -level normal-form verifier,
 $T(n) = (2^{\lambda n})^\mu$, $Q(n) = (\lambda n)^\mu$,
 $\text{TIME}_D(n) \leq T(n)$, $\text{TIME}_S(n) \leq Q(n)$;
 PROVE: `AnswerReduce(V)` has level $\max\{\ell + 2, 5\}$,
 the stated polynomial runtimes and directed completeness,
 soundness, and entanglement implications.

This is `thm:ar` in `ground-truth/gt-10-answer-reduction.tex`. The finite guarded decider is executable; the quantum implication, detyping, and asymptotic theorem are CITED.

5.4 The structural hypothesis

Introspection is the cited verifier transformation that makes encoded prover-side measurement information available to a smaller verifier while preserving a directed soundness contract.²⁸ *Anchoring* inserts special anchor questions with positive probability; *parallel repetition* runs several copies with an AND acceptance condition against possibly correlated strategies. Their combined repetition contract is cited from `thm:repetition` in `ground-truth/gt-11-parallel-repetition.tex`.

The relevant closure equations are

$$\text{Introspect}(\mathcal{Q}) \subseteq \mathcal{Q}, \quad \text{PCPCompose}(\mathcal{Q}) \subseteq \mathcal{Q}, \quad \mathcal{Q} \times \mathcal{Q} \subseteq \mathcal{Q}.$$

Concatenation, direct sum, the shared-seed distribution, and product are visibly CL-preserving in the base inductive datatype. Affine restriction is linear after earlier stages choose a direction, and random-point identity tests are pushforwards of a shared uniform seed. Typed graph choice is external, and the cited detyping transformation moves it back into two further CL levels.

This does not characterize every admissible query distribution, prove that a generic PCPP can be expressed in the algebra, or prove closure under the full introspection compiler. The structural statement is exactly C7: CONJECTURE. Introspection soundness and normal-form closure are not implemented. The present datatype makes several local closure laws legible, not the global compression theorem.

²⁶Ground truth: `sec:orac-def` in `ground-truth/gt-09-oracularization.tex`.

²⁷Ground truth: `fig:decider-pcp` in `ground-truth/gt-10-answer-reduction.tex`.

²⁸Ground truth: `thm:introspection` in `ground-truth/gt-08-introspection.tex`.

6 Correspondence tables

The evidence grade describes the local certificate leaf. The claim column is the current status in `claims/CLAIMS.md`; it can remain weaker than a certificate label when an adversarial verdict has not promoted the claim. `CONSTRUCTED` means enforced by a constructor, `CHECKED` means deterministic replay against an attached object, and `CITED` means imported from the named ground-truth statement.

6.1 Description front end (planned TB3)

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
<code>sec:tms</code> <code>gt-03-prelim.tex</code>	Canonical descriptions, hardwiring, universal simulation, timeout	<code>Quoted{A}</code>	<code>Quote, Eval, Specialize</code>	CITED; absent	none
<code>def:decider</code> <code>gt-05-games-normalform.tex</code>	Total five-input decision machine	<code>Quoted{Decider}</code>	<code>description_size</code>	CITED; absent	none
<code>prop:standard-succinct-sat</code> <code>gt-10-answer-reduction.tex</code>	Bounded trace to succinct 3SAT	<code>BoundedTrace, Succinct3SAT</code>	<code>bounded_trace, cook_levin</code>	CITED; absent	none
<code>sec:succinct-deciders</code> <code>gt-10-answer-reduction.tex</code>	Three shared reads become five independent blocks	<code>SuccinctDecoupled5SAT</code>	<code>decouple5</code>	CITED; absent	none

6.2 Polynomial rung (TB0)

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
<code>def:tseitin</code> <code>gt-10-answer-reduction.tex</code>	Circuit becomes a linear-size formula with auxiliary wires	<code>Circuit, TseitinFormula</code>	<code>tseitin, occurrences</code>	CHECKED occurrence	C2 CONJ.
<code>def:formula-arithmetization</code> <code>gt-10-answer-reduction.tex</code>	Boolean connectives become a formal polynomial	<code>Poly</code>	<code>arith_q, actual_degrees</code>	CHECKED	C8 TESTED
<code>sec:ld-encoding</code> <code>gt-03-prelim.tex</code>	Assignment becomes its multilinear extension	<code>Poly</code>	<code>g_a, dependency_blocks</code>	CONSTRUCTED/ CHECKED	C3 TESTED
<code>prop:zero-basis</code> <code>gt-10-answer-reduction.tex</code>	Divide by $z_i(1 - z_i)$ and require zero remainder	<code>ZeroDecomposition</code>	<code>zero_basis_decompose, verify_zero_decomposition</code>	CHECKED	C2 CONJ.
<code>fig:pcpverifier</code> <code>gt-10-answer-reduction.tex</code>	Check formula and zero identities at one view	<code>PCPView</code>	<code>pcpverifier</code>	CHECKED finite views	C1 CONJ.
<code>thm:pcp-decider</code> <code>gt-10-answer-reduction.tex</code>	Acceptance $> 1/2$ decodes an accepting trace, assuming low degree	<code>PCPPProof</code>	<code>build_pcp, parameter_policy</code>	CITED/ CHECKED parts	C5 SKETCH

6.3 CL and low-degree-test rung (TB1)

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
def:cl-func gt-04-cl.tex	CL functions are inductive conditional linear stages	AbstractCL	CLZero, CLStep, level	CONSTRUCTED	C4a TESTED
lem:cl-kth gt-04-cl.tex	A marginal is the sum of the first k stages	CLMarginal	marginal_k, sum_stage_outputs	CHECKED	C4a TESTED
lem:cl-concat, lem:cl-func-prod gt-04-cl.tex	Concatenation adds levels; direct sum takes a maximum	AbstractCL	concatenate, direct_sum, product	CONSTRUCTED	C7 CONJ.
lem:alnf, lem:dlnf gt-07-ldt.tex	CL pushforwards equal the line/point distributions	CLDistribution	L_Point, L_ALine, L_DLine, histogram	CHECKED TB1	C4a TESTED
fig:ld-decider gt-07-ldt.tex	Point/line answer consistency and degree format	LDPParams	ld_decider, restrict	CHECKED fixture	C4a TESTED
lem:ld-soundness gt-07-ldt.tex	Successful quantum strategy yields polynomial measurements	theorem leaf	no proof checker	CITED	C5 SKETCH

6.4 Typed answer-reduction rung (TB2)

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
def:typed-sampler gt-06-types.tex	A type graph selects a pair of CL maps sharing one seed	TypedSampler	TypedSampler, sample, pad_level	CONSTRUCTED base	C4b CONJ.
sec:ld-compiler gt-10-answer-reduction.tex	Six low-degree copies form an 18-type family	PCPCLMap	pcp_sampler, intrinsic_pcp_levels	local CHECKED; held	C4b CONJ.
sec:ld-compiler gt-10-answer-reduction.tex	Product with the oracularized sampler has 54 types	TypedProductCL	oracularize_sampler, typed_sampler_product	CHECKED cited	C4b CONJ.
fig:decider-pcp gt-10-answer-reduction.tex	Five guarded checks connect input, proof, low degree, and PCP	TypedAnswer ReducedDecider	typed_answer_reduced_decider	CHECKED finite paths	no current row
lem:detying-verifiers gt-06-types.tex	Remove types at cost +2 levels and 16^{54} soundness factor	CitedDetyed Verifier	detype	CITED	C4b CONJ.
thm:ar gt-10-answer-reduction.tex	Conditional complexity, completeness, soundness, entanglement contract	TypedAnswer ReducedVerifier	answer_reduce_pcp, AnswerReduce	CHECKED/ CITED	C5 SKETCH

6.5 Midpoint diagnostic (TB0.5)

Analytic anchor	Analytic statement	IR sort	Implementation	Grade	Claim
handoff.md midpoint diagnostic	False level- n claim has value $1 - 2^{-n}$ under the stated orbit condition	Test/Ask/Coin terms	toys/midpoint	proof plus exact tests	C6 PROVED

Analytic anchor	Analytic statement	IR sort	Implementation	Grade	Claim
<code>toys/midpoint/PROOF.md</code>	Sequential r -copy AND value is $(1 - 2^{-n})^r$; constant gap costs $\Theta(2^n)$ copies	exact dynamic program	<code>sequential_value</code>	proof plus exact tests	N1 PROVED

6.6 Compression and fixed point (planned TB4)

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
<code>fig:compress</code> <code>gt-12-compression.tex</code>	Repeat $\circ$ AnswerReduce $\circ$ Introspect	ClosedProgram Compressor	Compress	CITED; absent	C7 CONJ.
<code>thm:compression</code> <code>gt-12-compression.tex</code>	Nine-level gap-preserving compression with dimension bound	contract tree	no current implementation	CITED	C7 CONJ.
<code>fig:halt_f</code> <code>gt-12-compression.tex</code>	Explicit self-application constructs its own decider description	PartialProgram, Quoted	Hole, Specialize, Fix/YCode	CITED; absent	none

7 What is analytic and what is computed

7.1 The evidence boundary

The analytic content is the sequence of transformations and the implications attached to them. The computer currently realizes a strict middle segment.

Class	Present content	What would be required to move the boundary
Executed constructions	Circuit/Tseitin syntax; formula arithmetization; multilinear extensions; c_0 ; zero-basis quotients; point/line maps; typed products; guarded TB2 decider	Scale-independent front end for quoted programs and a constructor-faithful lazy CL stage representation for the PCP sampler
Certified identities	Normalized coefficient equality; zero remainder; structural and actual degrees; dependency blocks; CL marginals; TB1 exact histograms; finite parser and branch checks	General semantic certificates connecting trace, SAT, Tseitin, and arithmetization, plus promotion through adversarial verdicts
Cited analytic leaves	Cook–Levin asymptotics; general Tseitin semantics; PCP soundness; low-degree extraction; oracularization; detyping; introspection; rigidity; anchored repetition; compression	Formal proofs over quantified machines, distributions, Hilbert spaces, measurements, approximation metrics, and asymptotic bounds—not more finite samples

Lean or another proof assistant could check the algebraic core once finite fields, sparse polynomials, interpolation, and polynomial division are formalized. That would move the general zero-basis identity and degree lemmas from prose to proof terms. It would not by itself certify `lem:ld-soundness`:

that leaf requires a library for finite-dimensional Hilbert spaces, POVMs, state-dependent distances, tensor-code tests, and the quantitative extraction argument. Rigidity and oracularization require the same operator layer. Detying additionally needs a formal probability and rejection-sampling argument with the $16^{|\text{Type}|}$ loss.

Anchoring and parallel repetition are combined in `Repeat` and are theorem-backed rather than executed here. Introspection and compression would require a still broader object: a formal semantics for quoted verifier descriptions, resource bounds under specialization and universal evaluation, and a proof that the directed value and entanglement implications compose. The relevant final statements are `fig:compress` and `thm:compression` in `ground-truth/gt-12-compression.tex`.

7.2 The midpoint diagnostic

The recursive midpoint verifier for $y = f^{2^n}(x)$ asks for z and checks one of

$$z = f^{2^{n-1}}(x), \quad y = f^{2^{n-1}}(z)$$

with a fresh fair coin. It has a compact fixed-point description and perfect completeness. Under the orbit-prefix condition stated in C6, the exact optimum for a false claim is nevertheless

$$p_n = 1 - 2^{-n}.$$

Claim C6 is PROVED. For adaptive sequential AND repetition, N1 is also PROVED and gives value $(1 - 2^{-n})^r$; reaching error at most $1/2$ requires $r = \Theta(2^n)$, hence $\Theta(n2^n)$ transcript cost in the stated unit-cost model. Neither claim concerns quantum parallel repetition or the actual `Compress` operator.

The diagnostic isolates the point of the whole construction. A recursive term can represent an exponentially long computation, and a fixed point can make that term self-referential, while the acceptance gap collapses too quickly for iteration. Representability is easy; an iterable, gap-preserving analytic transformation is the hard requirement.

7.3 Final accounting

The lambda reformulation removes none of the mathematically difficult leaves. It makes quotation, hardwiring, description size, timeout, and self-reference explicit in one syntax. The polynomial IR makes the Cook–Levin-to-PCP middle visible as actual algebra, and the CL datatype exposes several sampler closure laws. What remains unchanged is the proof that high success in the typed quantum game yields consistent low-degree polynomial measurements, survives oracularization and detying with the stated quantitative loss, survives introspection and anchored parallel repetition, and composes into the gap-preserving compression theorem. Those are analytic theorems, presently CITED; they are neither consequences of homoiconicity nor conclusions of the finite computations reported here.